Embedded Linux & System Software Interview Handbook

Chapter 7 - Linux Kernel Synchronization & Concurrency (Part 1)

Topics: Concurrency Fundamentals, Race Conditions, Critical Sections and Synchronization Primitives

7.1 Why Synchronization is Required

Modern Linux systems execute multiple processes, kernel threads, interrupt handlers and workqueues concurrently. Without synchronization, shared data structures can become corrupted, leading to crashes, data loss or unpredictable behaviour.

Sources of Concurrency

Execution Context	Examples
User Processes	Multiple applications
Kernel Threads	kworker, kswapd
Interrupt Context	GPIO, UART interrupts
SoftIRQ / Tasklets	Networking
Workqueues	Deferred driver work
Multiple CPUs	SMP systems

7.2 Race Condition

A race condition occurs when two or more execution contexts access shared data simultaneously and at least one modifies it. The program's behaviour then depends on the timing of execution.

Critical Section

A critical section is a region of code that accesses shared resources and therefore must be protected from concurrent execution.

Common Synchronization Primitives

Primitive	Typical Usage
Spinlock	Short critical sections, interrupt context
Mutex	Sleeping locks for process context
Semaphore	Resource counting
Atomic Variables	Simple counters and flags
Completion	One-time synchronization
Wait Queue	Waiting for an event

Choosing the Correct Primitive

Use a spinlock when the protected code executes quickly and cannot sleep. Use a mutex when the code may block or sleep. Atomic operations are suitable for simple counters or flags that require lock-free updates.

Common Bugs

- Race conditions
- Deadlocks
- Priority inversion
- Double locking
- Missing memory barriers

Interview Questions

1. What is a race condition?
2. Define a critical section.
3. Why is synchronization required in the Linux kernel?
4. When would you use a spinlock instead of a mutex?
5. What are common concurrency bugs in kernel code?

Key Takeaways

Understanding concurrency is fundamental to Linux kernel development. Selecting the appropriate synchronization primitive is essential for writing correct, scalable and high-performance drivers.